Studio Nx

백엔드 235 — 이펙트 칸

2026-10-18

Backend(FastAPI/Python) 235 — `track_service` 이펙트 칸 (체인 순서와 갈래 자리)

항목	내용
파일	`studionx/backend/app/services/track_service.py`
작업	S1-09 기본 악기·이펙트: 이펙트 칸 읽기·붙이기·고치기·떼기 (PRD 02 FR-263, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	234 악기 칸 · 236 연결선

1. 하는 일

트랙 안의 이펙트 칸(`track_plugins`)을 다루는 함수 다섯 개를 둡니다.

함수	하는 일	권한
`list_effects`	트랙의 이펙트 목록 (칸 번호 순 = 체인 순서)	멤버면 누구나
`add_effect`	이펙트 한 칸 붙이기. `EFFECT` 플러그인만 가능	OWNER, EDITOR
`update_effect`	칸 번호(순서), 카드 자리(`node_x`·`node_y`), 파라미터, 바이패스 고치기	OWNER, EDITOR
`remove_effect`	이펙트 칸 떼기. 그 칸에 이어진 연결선도 DB CASCADE 로 함께 지워집니다	OWNER, EDITOR
`_require_effect`	경로의 트랙에 속한 이펙트인지 확인하고 쓰기 권한까지 검사	OWNER, EDITOR

이펙트 칸은 **EFFECT 플러그인만** 들어갑니다. `slot_index` 가 소리가 지나가는 순서이고, `node_x`·`node_y` 는 갈래(패처) 화면에서 카드를 놓은 자리입니다 (FR-269).

2. 언제 쓰나

쓰는 곳	
`GET/POST /tracks/{id}/plugins`, `PATCH/DELETE /tracks/{id}/plugins/{pid}` (240)	네 라우터가 이 함수들을 부릅니다
화면의 [Effects] 칸	체인 카드를 붙이고 끌어 순서를 바꾸고 파라미터를 저장할 때 부릅니다 (FR-263)
`add_link` (236)	연결선이 가리키는 이펙트가 같은 트랙의 것인지 확인할 때 씁니다
S1-10 저장·불러오기	이펙트 구성과 순서, 갈래 자리를 함께 저장하고 되살립니다 (FR-254)

3. 코드

목록과 한 칸 찾기

```
def list_effects(db: Session, user: User, track_id: int) -> Sequence[TrackPlugin]:
    """트랙의 이펙트 칸 목록 (slot_index 순 = 체인 순서). 멤버면 누구나 (FR-263)."""
    track = require_track(db, user, track_id)
```

```
    stmt = (
        select(TrackPlugin)
        .where(TrackPlugin.track_id == track.track_id)
        .order_by(TrackPlugin.slot_index, TrackPlugin.track_plugin_id)
    )
    return db.execute(stmt).scalars().all()


def _require_effect(db: Session, user: User, track_id: int, track_plugin_id: int):
    tp = db.get(TrackPlugin, track_plugin_id)
    if tp is None or tp.track_id != track_id:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Track effect not found.")
    _require_editor_track(db, user, track_id)
    return tp
```

붙이기

```
def add_effect(db, user, track_id, *, plugin_id, slot_index, node_x=None, node_y=None,
               params=None, is_bypassed=False):
    """트랙에 이펙트 한 칸 붙이기 (FR-245·263). OWNER·EDITOR 만, EFFECT 만 가능."""
    _require_editor_track(db, user, track_id)
    plugin = plugin_service.require_plugin(db, plugin_id)
    if plugin.plugin_type != PluginType.EFFECT:
        raise AppError(ErrorCode.BAD_REQUEST, 400, "Only EFFECT plugins can be added to effects chain.")
    _require_free_slot(db, TrackPlugin, track_id, slot_index, "effect")
    tp = TrackPlugin(track_id=track_id, plugin_id=plugin_id, slot_index=slot_index,
                     node_x=node_x, node_y=node_y, params=params or {}, is_bypassed=is_bypassed)
    db.add(tp)
    db.commit()
    db.refresh(tp)
    return tp
```

고치기 (순서 바꾸기 포함)

```
def update_effect(db, user, track_id, track_plugin_id, **kwargs):
    tp = _require_effect(db, user, track_id, track_plugin_id)
    if "slot_index" in kwargs and kwargs["slot_index"] is not None:
        _require_free_slot(db, TrackPlugin, track_id, kwargs["slot_index"], "effect",
                           exclude_id=tp.track_plugin_id)
    for key, value in kwargs.items():
        if value is not None and hasattr(tp, key):
            setattr(tp, key, value)
    db.commit()
    db.refresh(tp)
    return tp
```

문법 풀이

식	뜻
`slot_index: int` (기본값 없음)	이펙트는 순서가 있어야 하므로 붙일 때 반드시 받습니다. 악기 칸과 다른 점입니다
`_require_free_slot(db, TrackPlugin, ...)`	악기와 같은 도우미를 이펙트 모델로 부릅니다 (234 §3)

식	뜻
`exclude_id=tp.track_plugin_id`	자기 자리로 다시 보내는 요청은 통과시킵니다
`"slot_index" in kwargs`	순서를 바꾸는 요청일 때만 칸 검사를 합니다. 파라미터만 바꿀 때는 검사하지 않습니다
`params or {}`	붙일 때 파라미터를 안 보내면 빈 사전입니다. 나중에 화면이 기본값으로 채웁니다
`hasattr(tp, key)`	모델에 있는 칸만 바꿉니다. `track_id` 같은 칸을 손으로 바꾸는 요청을 막습니다

4. 왜 이렇게

선택	이유
칸 번호가 곧 체인 순서	CHAIN 방식은 소리가 `slot_index` 차례로 지나갑니다. 따로 순서 칸을 두지 않아도 됩니다 (FR-263·267)
순서 바꾸기를 `PATCH` 하나로	카드를 끌어 옮기는 일은 "그 카드의 칸 번호를 바꾼다" 로 표현됩니다. 화면은 옮긴 카드 한 장만 보내면 됩니다
이미 찬 칸이면 400	두 이펙트가 같은 번호를 가지면 순서를 정할 수 없습니다. DB UNIQUE 를 서비스가 먼저 확인합니다 (234 §4)
자리(`node_x`·`node_y`)를 그대로 저장	갈래 화면을 다시 열어도 카드가 놓인 자리가 같아야 합니다 (FR-269)
떼면 연결선도 함께	선만 남으면 없는 카드를 가리킵니다. DB CASCADE 로 함께 지웁니다
`is_bypassed` 를 따로	소리 처리만 잠시 건너뜁니다. 카드는 그대로 남아 순서와 설정을 지킵니다

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 붙이기 201과 응답 칸 이름, `node_x` 저장, `is_bypassed` 기본값 false
- 악기를 이펙트 자리에 넣으면 400
- 목록 개수와 고치기(바이패스, 칸 번호 2로 옮기기)
- 같은 칸 두 번은 400, 남이 쓰는 칸으로 옮기기도 400
- 떼기 204

> 슬롯을 서로 맞바꾸는 화면 동작(예: 1번과 2번 교환)은 지금 API 로는 한 번에 안 됩니다. 한 카드를 먼저 빈 칸(예: 100번)으로 옮기고, 다른 카드를 원래 자리로 옮긴 뒤, 첫 카드를 마지막 자리로 옮기는 세 번의 `PATCH` 로 처리합니다 (FR-263 순서 편집).